AURELITH Operating Blueprint

Version: 0.1 — decision draft

Purpose: Turn the AURELITH Studio → State → Stage concept into a narrow, operable browser product that a creator can author and a second operator can load, inspect, recover, and benchmark.

Status: The required control surfaces and build order below are **targets** until they pass the stated gates. No line item constitutes a supplier quote, revenue forecast, or a claim of physical-output readiness.

> **The v1 artifact is a portable scene manifest and a controlled handoff—not a general-purpose DAW, VJ platform, MIDI host, haptic instrument, or laser controller.**

1. Product decision: what has to work first

The user-facing product should make one promise exceptionally clear: an author can configure **one bounded audiovisual scene**, and a second operator can re-create the approved state from a readable manifest without needing the author to explain the underlying code. This preserves the existing product contract and confines risk to a measurable reference environment.

User	Job	Required outcome	Explicitly out of scope for v1
Author / creative technologist	Shape a scene and save a reliably understood state.	Bounded macro changes, previewable patches, named audio/visual behavior, and an exported manifest.	A general DAW, arbitrary shader/code generation, multi-scene library.
Stage operator / collaborator	Load, inspect, confirm readiness, and recover.	Scene revision, device status, control map, quality tier, warning list, and a safe fallback.	Hidden “magic” automation, author-dependent troubleshooting.
Technical lead	Decide whether the reference setup is acceptable before rehearsal.	Benchmark record with source, units, measurement window, and limitation labels.	A venue-certification or universal-latency guarantee.

The browser should remain the **control and evidence shell**. Audio scheduling belongs to the audio clock, visual animation belongs to the render loop, and the manifest is the only persistent cross-person contract. Web Audio is designed around an audio routing graph and supports scheduled playback, parameter automation, channels, analysis, and audio worklets; it is the appropriate primitive for the bounded runtime. [1] [2]

2. User-facing experience: controls before features

The operating surface should be intelligible in a dark venue and useful in rehearsal. It needs a small, stable set of controls with a visible target state, a visible applied state, and a plain-language result. The objective is **operator confidence**, not control density.

2.1 The three-mode interface

Mode	Audience	Visible controls	Non-negotiable feedback
Studio	Author	Eight macros, semantic patch proposal, patch diff, scene revision, save/export.	Current vs target values, undo path, patch confidence labelled as a simulation until validated.
Stage	Operator	Four protected performance macros, quality tier, transport/readiness, “stabilize,” and manifest transfer.	What is loaded, what is connected, what is missing, and exactly what fallback is active.
Diagnostics	Technical lead	Device list, MIDI permissions, audio-context state, FPS/frame-time, quality transitions, error ledger, export record.	Measurement source, sample window, unit, browser/OS/device record, and “not measured” where appropriate.

The **Studio** view may expose all eight scene macros because it is for deliberate authorship. The **Stage** view should expose only four mappings that are rehearsed and named in performance language: **Intensity**, **Depth**, **Atmosphere**, and **Focus**. Material, narrative weight, and every risky/systemic parameter remain manifest-governed rather than

continuously adjustable. The optical macro stays locked at `simulation-only` in every v1 user interface.

2.2 Macro contract

Every macro should declare its human meaning, range, smoothing behavior, audio targets, visual targets, and fallback behavior. Avoid ambiguous “magic knobs.” A macro does not write directly to a renderer or a device; it requests a bounded target-state update.

Macro	Stage access	Audio target	Visual target	Mapping rule	Fallback
Intensity	Yes	Master-safe perceived energy via gain ceiling and transient density.	Emissive strength, motion amplitude, contrast.	0–1; gain is clamped below a declared ceiling.	Reduce visual energy before touching audio.
Depth	Yes	Filter cutoff / reverb send within approved bounds.	Camera parallax, fog distance, layer separation.	Smooth, slow; no hard scene jump.	Hold previous visual state.
Atmosphere	Yes	Noise bed, granular density, or reverb mix.	Particle density, haze, texture modulation.	Smoothed; quality tier caps particle cost.	Disable secondary particles first.
Focus	Yes	Band emphasis or transient weighting.	Central object sharpness, vignette, cue emphasis.	Centre-biased; no audio pan change without explicit mapping.	Preserve current view.
Material	Studio	Timbre colour / drive amount.	Surface roughness, iridescence, shader branch.	Studio-only until rehearsed.	Return to approved material preset.
Sacred ↔ Industrial	Studio	Harmonic colour / sample layer mix.	Geometry language and light temperature.	Quantized only at transport boundary.	Hold last valid preset.

Optical / Simulation	Locked	None.	Simulated beam or abstract energy field only.	Fixed in v1.	Always remain simulation-only.
Narrative Weight	Studio	Arrangement layer density.	Cue hierarchy and transition duration.	Slow, scene-level parameter.	Defer to manifest default.

2.3 MIDI design: direct, visible, recoverable

MIDI should be optional, never silently assumed. The Web MIDI API is restricted to secure contexts, may be constrained by the `midi` Permission Policy, and is not Baseline across widely used browsers; therefore the product must ship with keyboard/pointer fallback and a prominent compatibility check. [3]

MIDI capability	v1 behavior	Acceptance criterion
Device discovery	On explicit **Connect MIDI** action, list inputs/outputs with name, manufacturer, state, and permission outcome.	Operator sees “connected,” “unavailable,” or “permission denied,” never a blank surface.
Learn mode	Author selects a macro, moves one hardware control, confirms the resulting CC/note mapping.	Mapping is stored in manifest with device-agnostic logical ID plus optional device fingerprint.
Stage mapping	Stage view loads only the approved four protected macros.	Incoming CC changes the named target state and visible value.
Note triggers	Limit to cue/preset recall in v1; no note is allowed to fire a physical output path.	Trigger event is recorded in the session ledger.
Disconnect/reconnect	Freeze last target state; show non-blocking warning; require explicit rebind when device identity changes.	Audio/visual runtime remains stable after disconnect.
MIDI output	**Not required for v1.**	No feedback LED protocol or external timing promise before demand is established.

Web MIDI represents inputs and outputs through `MIDIAccess`, port maps, message events, and connection-state events. It can queue output messages, but AURELITH should deliberately avoid placing output feedback or transport-critical dependence in the first pilot. [3]

3. Audio architecture and sound-effects policy

3.1 Audio is a graph, not a UI effect

Use one `AudioContext`, one scene routing graph, and one final safety bus. The W3C model supports modular routing, dynamics compression, filters, convolution, channel split/merge, and analysis; `AudioWorklet` is the appropriate custom-processing boundary, while legacy `ScriptProcessorNode` is deprecated due to main-thread performance limitations. [1] [2]

Plain Text

```
MIDI / keyboard / pointer
        ↓
Input normalizer → bounded target state → state interpolator
        ↓                                      ↓
     event ledger                          audio parameter plan
                                                ↓
Sources → per-source gain → per-source FX send → scene buses → safety bus → des
                                                ↓
                                          analyser tap
                                                ↓
                                       visual feature frame
```

Audio channel/bus	Purpose	Required controls	Guardrail
Music input	Optional routed track/media or generated source.	Input gain, sidechain analysis enable.	User must select source; do not auto-open microphone input.
Scene tone	Synth/drone/texture supporting the visual field.	Intensity, depth, material.	Cap voice count and use a stable voice-steal rule.
Cue / one-shot	Transition confirmation, scene cue, reset acknowledgement.	Cue level only.	Avoid frequent non-essential UI sounds; include mute.

Atmosphere	Noise, granular, or ambient layer.	Atmosphere, narrative weight.	Quality tier reduces density before it removes core rhythm.
Master safety bus	Final gain, optional compressor/limiter, analyser tap.	Output trim, mute, diagnostics.	No control may bypass it.

Sound effects should communicate meaningful state changes only: manifest validated, safe fallback applied, cue recalled, and input disconnected. Do not add decorative clicks to every macro movement. Each effect requires a **mute control**, a level relative to the master bus, and a declared source/license record. No audio is initiated until an intentional user gesture activates the audio context; Tone.js likewise documents that scheduling before `Tone.start()` resolves can produce silence or incorrect scheduling. [4]

3.2 Scheduling and transport

Use the audio clock as source of truth for scheduled audible events. `AudioContext.currentTime` is the rendering graph’ s timeline and does not inherently synchronize to every other system clock; render/UI clocks must be bridged rather than treated as identical. [1] Tone.js is an acceptable thin scheduling layer if its global transport and signal model reduce implementation risk; its own guidance warns that JavaScript callbacks are not precisely timed and exposes an audio-time value for event scheduling. [4]

Concern	Decision	Why it is correct for v1
Timebase	`AudioContext.currentTime` / Tone transport for audio events.	Audio scheduling is the only clock with a musical timing contract.
UI updates	React state at low frequency only.	Prevents React rerenders from becoming the live control loop.
Visual state	Interpolate toward audio-derived feature frames in requestAnimationFrame/WebGL.	Keeps visuals smooth while accepting display-rate limits.
Transport sync	Use explicit beat/bar timestamps, not “whatever frame arrived.”	Enables cue recall and deterministic benchmark records.
Offline rendering	Optional later via `OfflineAudioContext`.	Useful for scene preview/export, not required to prove the handoff wedge.

4. Synchronous audiovisual state: the control contract

The system needs a **single semantic state owner**. Inputs may request changes and observers may report metrics, but neither has permission to mutate the runtime directly. This avoids the common failure where MIDI, UI, visual code, and telemetry each become competing sources of truth.

4.1 State layers

Layer	Owner	Update rate	Persisted	May write audio?	May write visual?
Intent event	Input normalizer	Event-driven	Yes, ledger	No	No
Target state	State reducer	Event-driven	Yes, manifest/revision	No	No
Runtime state	Interpolator / audio plan	Audio/frame cadence	No	Yes, bounded parameters	Yes, feature frame only
Observation	Diagnostics sampler	Fixed sample window	Yes, benchmark record	No	No
Output adapter	Future extension	Explicitly gated	Separate record	Not v1	Not v1

The operator should always see these distinct labels in the interface: **Target**, **Applied**, **Measured**, **Simulation**, and **Future option**. A green cue means “the target was accepted by the bounded runtime”; it does not mean that physical latency, a haptic effect, or a laser path has been certified.

4.2 Audio-to-visual features

For v1, expose a small feature vector rather than an unbounded FFT stream:

JSON

```
{
  "tAudio": 0,
  "transport": {"bpm": 120, "beat": 1, "bar": 1},
```

```
  "features": {
    "rms": 0.0,
    "low": 0.0,
    "mid": 0.0,
    "high": 0.0,
    "onset": 0.0
  },
  "target": {"intensity": 0.64, "depth": 0.58, "atmosphere": 0.38},
  "quality": "HIGH"
}
```

An `AnalyserNode` can provide real-time frequency and time-domain data for visualization; treat this as a visual-control signal, not a proof of subjective audio quality. [2] Visual animation uses `requestAnimationFrame`, whose cadence generally follows display refresh rate and pauses in many background-tab scenarios. MDN notes that its precision relative to an external clock such as Web Audio is bounded by a frame—16.67 ms at 60 Hz. [5] Consequently, the visualizer needs **perceptual synchrony targets**, not an unsupported "sample-accurate visual" claim.

Visual system	Primary signal	Smoothing	Quality policy
Macro deformation	Target state	80–500 ms depending on macro.	Preserve movement; simplify material.
Beat/onset cue	Transport + onset flag	One beat window / controlled decay.	Preserve cue hierarchy.
Material modulation	Low/mid/high bands	50–250 ms EMA.	Reduce post-processing, not signal meaning.
Particle/field density	RMS + atmosphere	200–600 ms.	Cap count/instances first.
Scene transition	Manifest cue timestamp	Audio-clock start, visual frame interpolation.	Hold prior valid frame if new asset not ready.

5. Dependency choices: use less, own the contract

Domain	Recommended v1 choice	Add only when	Avoid in v1

Web audio	Native Web Audio plus **Tone.js** only if transport/sequencing materially shortens build time.	Scene needs scheduled cues, synths, or musical transport.	Multiple competing audio wrappers.
MIDI	Native Web MIDI behind an adapter interface.	Browser/device support confirmed in reference environment.	A controller-vendor SDK or MIDI output dependency.
Visual runtime	`three` + React Three Fiber only for the actual scene; plain CSS/Canvas for dashboard UI.	Subsurface Cathedral becomes interactive 3D, not only an editorial visual.	React rerendering every animation frame.
Visual utilities	`@react-three/drei` selectively.	It removes measurable boilerplate such as adaptive performance or asset loading.	A large effect stack before a baseline frame budget exists.
State	Small external store such as **Zustand**, partitioned by intent/target/runtime/observation.	Cross-component state must avoid React context churn.	One global object mutated by UI, MIDI, and renderer.
Schemas	`zod` plus JSON Schema export/check.	Every manifest and patch boundary.	Untyped JSON handoffs.
Metrics	Browser Performance API + custom event ledger; export JSON.	Benchmark cases require raw recordings.	Claiming physical audio latency from frame samples.
Persistence	Local-only manifest first; optional user storage later.	Real pilot needs version history/share.	Database work before a second-operator transfer succeeds.

React Three Fiber documentation recommends on-demand rendering for scenes that come to rest, resource reuse, instancing, and adaptive performance degradation; its `PerformanceMonitor` is designed to feed quality changes when average FPS crosses bounds. [6] The recommended position is therefore **audio-first progressive degradation**: preserve audio availability and accepted macro meaning, then lower pixel ratio, disable post effects, reduce particle count, and finally simplify materials. Zustand is appropriate only as a small selector-based state store, not as the audio or visual clock. [7]

6. Performance and quality contract

These are proposed **reference-environment targets**, not guarantees. Use a declared browser, OS, CPU/GPU, display mode, audio device/path, power mode, scene revision, and control path in every report.

Metric	Measured method	Target gate	Failure action
Cold scene readiness	`performance.now()` from intentional load to readiness state.	Establish baseline first; then track regression no worse than 10%.	Preserve last known good build; inspect asset/initialization cost.
Visual frame time	`requestAnimationFrame` median/p95 in a 10-minute deterministic soak.	**Target:** p95 ⩽ 25 ms in HIGH; p95 ⩽ 33.3 ms in SAFE.	Step down visual quality; keep audio path unchanged.
Frame stability	Worst-frame/event ledger.	**Target:** no repeated >100 ms freeze without recorded cause.	Disable expensive material/effect branch; trace allocation.
Audio availability	AudioContext state, output-device record, activation event.	Running after explicit gesture and remains running during stated test.	Show blocked/unavailable state; no false “audio ready.”
Audio scheduling	Scheduled cue time vs audio clock; offline/rehearsal listening test.	No scheduling claim until controlled test passes.	Rework scheduling; do not replace with UI timers.
MIDI input reaction	Input event timestamp → target-state reducer timestamp.	**Target:** record p95 application time ⩽ 20 ms in the reference browser.	Show input lag warning; reduce event work/debounce policy.
Visual cue synchrony	Audio-clock cue marker → first rendered cue frame, recorded as video/trace.	**Target:** perceptually aligned in controlled rehearsal; report method and observation.	Adjust visual lead/lag calibration; do not claim a universal ms value.
Manifest transfer	Second operator completes declared checklist in clean reference environment.	100% completion with no author intervention for the pilot scene.	Improve manifest/readiness UI, not scene scope.

Recovery	Intentional missing display path or invalid macro test.	Error visible, runtime safe, recovery action understandable.	Block stage load until a valid state is restored.

7. Build order: prove the wedge before adding spectacle

Order	Increment	Deliverable	Must pass before next order
0	Product/environment freeze	One scene, reference laptop/browser/device/display/audio path, one controller family, versioned manifest schema.	Everyone can name what is inside/outside the test.
1	Control kernel	Keyboard/pointer macro panel; target/applied labels; patch review/undo; event ledger.	Bounded target updates can be accepted/rejected/recovered.
2	Audio kernel	User-activated AudioContext; one source path; master safety bus; analyser feature vector; mute.	Audio availability and cue scheduling are visible and stable.
3	Visual kernel	One material/geometry scene; feature-frame interpolation; HIGH/MEDIUM/SAFE policy.	10-minute benchmark completed with raw record.
4	MIDI adapter	Connect/disconnect, learn, mapping manifest, protected stage controls.	Device disconnect does not destabilize scene; mapping transfer is readable.
5	Stage handoff	Manifest load/validation, readiness checklist, transfer/recovery event log, browser-record export.	Second operator completes transfer without author help.

6	Pilot hardening	Regression suite, asset ledger, support sheet, controlled rehearsal.	Five relevant discovery signals plus one credible scoped pilot path.
7	Deferred experiments	Haptics comparative study, video capture/export, external control adapters.	Only after the core handoff and performance gates pass.

8. Contracts and handoffs

8.1 Artifacts that must exist

Artifact	Author	Consumer	Required content
`scene-manifest.json`	Authoring runtime	Operator/runtime	Scene/revision, macro values, MIDI logical map, quality policy, required paths, warnings, fallback.
`mapping-sheet.md`	Author / technical lead	Operator	Four stage controls, range, default, effect, panic/recovery behavior.
`benchmark-record.json`	Diagnostics runtime	Technical lead	Environment, methods, raw metric samples/summary, known limitations, manifest revision.
`operator-runbook.md`	Technical lead	Stage operator	Preflight, start, safe fallback, device reconnect, audio unlock, stop, escalation.
`asset-ledger.csv`	Artist / producer	Builder/operator	Asset source, license, format, size, optimization status, scene use.

`pilot-scope.md`	Producer / buyer	Team	Reference environment, support hours, acceptance tests, exclusions, pricing basis, change-control rule.

8.2 Handoff ceremony

The author exports a manifest and mapping sheet after approving the scene. The technical lead runs the benchmark with the exact manifest revision and attaches the raw record. The operator performs a clean-load test, intentionally simulates one failure, confirms the fallback language, and signs an acceptance note. Any change to macros, mappings, assets, audio path, browser version, or quality policy increments the manifest revision and requires another transfer test.

9. Budget-control model

No project budget was supplied. The following is therefore a **planning scenario**, not a quote: a blended delivery rate of **$80/hour**, a single scene, a browser-first proof, no venue labor, no show-day SLA, no custom hardware, and a **20% contingency**. The total is derived as $(140 + 180 + 160 + 100) \times \$80 \times 1.20$.

Workstream	Hours assumption	Base planning cost	What it buys	Stop condition
Control + manifest kernel	140	$11,200	Schema, target/applied state, patch/undo, export, operator labels.	No second-operator comprehension in early test.
Audio + MIDI kernel	180	$14,400	Web Audio graph, analyser features, MIDI adapter, keyboard fallback, mappings.	Reference controller/browser cannot produce stable mapped state.
Visual + benchmark kernel	160	$12,800	One scene, adaptive quality tiers, metrics,	10-minute soak or quality

			benchmark/recovery suite.	fallback repeatedly fails.
Handoff + pilot package	100	$8,000	Runbook, transfer flow, documentation, test rehearsal, evidence packaging.	No buyer/operator describes a costly handoff problem.
Base build	**580**	**$46,400**	Bounded browser proof.	—
Contingency	20%	$9,280	Browser/device variance and benchmark rework.	Release only against named risk.
Optional capped reference kit	N/A	Up to $1,500	One controller, audio interface/path, cables/adapters only after the control contract is stable.	No purchase until device criteria are written.
Planning ceiling with kit	—	**$57,180**	Evidence-first pilot-ready foundation.	Cap scope rather than exceeding ceiling.

The highest-leverage cost control is not a cheaper dependency. It is a strict rule: **do not build haptics, laser/optical outputs, multi-scene content, or venue-specific integration before the manifest handoff succeeds with a second operator and the reference benchmark is stable.**

10. Risks that should change the plan

Risk	Early indicator	Required response
MIDI incompatibility or permission refusal	`requestMIDIAccess()` unavailable/denied or device churn.	Make pointer/keyboard first-class; make MIDI an adapter, not a requirement.
Browser visual cost	Frame p95 misses tier target during soak.	Cut DPR/post effects/particles before changing macro semantics.

Audio unlock/availability	Context remains suspended or device path changes.	Provide explicit activation/preflight and re-check path; never silently retry output.
State divergence	MIDI/UI/visual disagree on a macro value.	Enforce single reducer, event ledger, and target/applied display.
Handoff ambiguity	Operator asks author for settings, mapping, or fallback explanation.	Improve manifest/runbook; do not add features.
Commercial ambiguity	Positive reactions but no actual workflow artifact or budget owner.	Continue discovery; do not call it pilot demand.
Physical-output scope creep	Requests for lasers or haptics before test evidence.	Keep adapters absent, labels explicit, and require independent safety/feasibility review.

11. Immediate next implementation increment

Build **Order 1 + the start of Order 2** as a single demonstrable panel:

1. Add a `Control Kernel` store with eight macro definitions, target/applied values, source (`pointer`, `keyboard`, `midi`), event timestamps, and undo.
2. Add an `Audio Kernel` with explicit user activation, one quiet synth/texture source, master mute, analyser feature-frame output, and a visible source label.
3. Add a **MIDI status card** that detects availability only after a user action, lists connected input ports, and falls back cleanly to keyboard/pointer.
4. Add a **sync inspector** showing audio clock, visual-frame timestamp, target vs applied macro values, and labels every non-physical value accurately.
5. Export these values into the existing manifest/benchmark record. Do not add an external output adapter.

References

[1]: W3C, *Web Audio API 1.1*

[2]: MDN, *Web Audio API*

[3]: MDN, *Web MIDI API*

[4]: Tone.js, *Web Audio framework, transport, scheduling, and audio activation*

[5]: MDN, *Window.requestAnimationFrame()*

[6]: React Three Fiber, *Scaling performance*

[7]: Zustand, *Introduction*